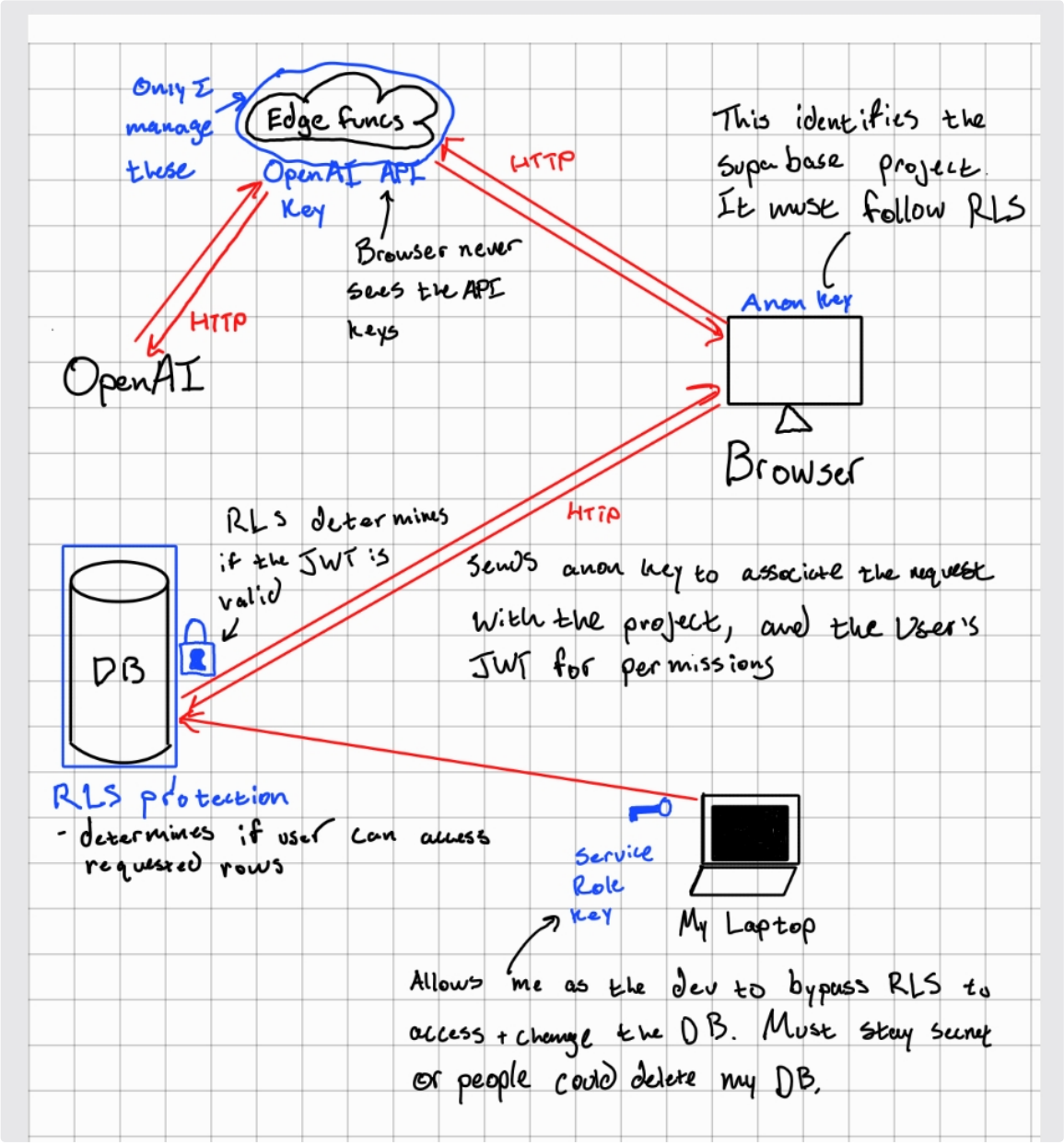
📝 Reflection Questions

1. Security Architecture ⭐⭐⭐⭐⭐

Draw or describe a diagram showing what protects each sensitive asset in this application. Your answer should address:

- Why is the anon key safe to expose in `config.public.json`?
- What would happen if someone got the service role key?
- How do Edge Functions protect the OpenAI API key?
- What role does Row Level Security play?



2. Edge Functions & the "Secure Middle" ⭐⭐⭐⭐⭐

Why can't we call OpenAI directly from the browser? Explain the flow: browser → Edge Function → OpenAI. What role does JWT verification play in this chain? How does this pattern apply to other production applications?

- Calling OpenAI always requires an API Key, regardlesss of where it is done. Any code run from the browser is public: someone can access the real values being used, not just parameter names. So any API call done from the browser will show the API key, which someone could then take and spend all of the money on it.
- The flow of browser -> Edge function (backend) -> OpenAI keeps the API key out of a public space: only I can manage and see the Edge function. The most that the browser ever knows is that an edge function is being called, it doesn't know anything about the API key.
- JWT verification comes into play in this chain using the shared `auth.ts` file. That is a shared function that executes before any Edge functions. That is where the JWT tokens are verified before executing the backend code.

- From my understanding, JWT token issuing and verification is a very common pattern for production applications, so this pattern applies in lots of places.

3. From SQL to Semantics ⭐⭐⭐⭐

Compare keyword search (SQL `ILIKE`) vs semantic search (vector cosine similarity):
Keyword search matches on exact phrases in the talks. If the phrase the user enters is similar to a real phrase in the talks, it doesn't matter, there won't be a match.
Semantic search comparse sentence embeddings with the embedding for the user's query. The most "similar" according to the cosine distance between the query vector and a talk's sentence vector is returned.

- When would each be better?
  - Keyword search is better when you are trying to get a quote or citation. For example if you know an exact phrase from a talk, but can't remember the speaker or the talk title, keyword search works well.
  - Semantic search is better when you are trying to gather any info on a concept. If you are preparing a talk for church about the temple and how it blesses us, putting "how the temple blesses us" into keyword search is only helpful if there are talks where the speaker says "how the temple blesses us". But semantic search can get an embedding for "how the temple blesses us", and get the closest passages to help provide useful content.
- Give a specific query example where one succeeds and the other fails
  - If I remember President Oaks telling a talk with a story about an Irish setter, but don't remember the name of the talk or the date it was given, I can go to keyword search and type "Irish setter" and that should find the right talk. Semantic search might do ok with this query, simply because the correct talk will have more chunks with "Irish setter" than other talks, but it will not be as precise as keyword search.
- What makes semantic search "understand" meaning?
  - Embeddings are what allow semantic search to understand meaning. Through math and machine learning techniques, the contents of the talks are run through a model to get a vector that represents what that content means (which is relevant only in the context of that model's embeddings). Then, how "similar" one thing is to another is based on the distance between two embedding vectors.
  - So, the results of a semantic search is content whose embedding vectors are the smallest distance from the embedding vector of the question.

4. RAG vs Fine-Tuning ⭐⭐⭐⭐

We used RAG instead of fine-tuning a model on conference talks. Research what fine-tuning is (this wasn't covered in class — go find out!). Then explain:
Fine-tuning is repeatedly retraining a model and adjusting its weights to alter the behavior and output.

- What are the trade-offs between RAG and fine-tuning?
  - RAG is better for answering from a certain source and providing citations. You are giving the model the source material to use, so it can easily based its answer off of that. RAG is not good at answering in a certain format or tone. That comes from the model's training, and RAG doesn't affect that at all.
  - Fine-tuning helps change how the model answers a question. Things like asking the user questions for understanding, providing disclosure on limitations, and more, is controlled through fine-tuning. Fine-tuning does not control what the content of the model's answer is very well. Fine-tuning essentially is what the model is studying on. Once the model has finished studying (been trained), if it can't look anything up (RAG), it relies on its memory from training, which can become fuzzy.
- Why did we choose RAG for this application?
  - We chose RAG because it is perfect for this use case: answering questions from a specific source material and providing citations from that material. That is one of the areas where RAG excells.
- When might fine-tuning be the better choice?
  - If we wanted every response to have a certain voice or format, such as using direct quotes rather than paragraphs, or following certain behaviors every time, fine-tuning would work better, because that response behavior would be deeply engrained in the model.

5. AI-Assisted Development ⭐⭐⭐

Briefly describe how your AI coding assistant helped you during this assignment:

- What did it do well?
  - It did great at writing the Edge functions and editing the python scripts. I would glance over the code, but it worked so consistently that I lost the need to review it's code output
  - It did a good job of explaining things that I asked about it, both general concepts and project specific questions.
- Where did you need to guide it?
  - Mostly in the scope of the plans it created. Often after giving it the description of what I wanted for an Edge function or something else, it would get the general idea, but sometimes add suggestions for extra details that I sometimes agreed with, but other times had to correct it and instruct it to ignore that detail or implement something else.
- What did you learn about working with AI tools?
  - I learned there is a lot of power in using them. It requires me to be a little uncomfortable and change how I think about coding and developing software, but there is a lot of potential if I can do that.
  - I also learned that Claude Code has some impressive differences to other models and harnesses. I was suprised at it's behavior, without any configuration by me: it asked occasional questions after coding something or drafting a plan to ensure I understood the project, it would outline what it did that was outside of our discussed spec, and more, all without me asking it to do so.